

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN

MODALIDAD A DISTANCIA

Trabajo Práctico Integrador (TPI)

Gestión de Datos de Países en Python: filtros, ordenamientos y estadísticas

Materia: Programación 1

Alumno:

Agustín Steinberg

Fecha de entrega: 20/07/2026

Índice

1. Marco teórico	3
1.1 Listas	3
1.2 Diccionarios	3
1.3 Funciones	3
1.4 Condicionales	3
1.5 Ordenamientos	3
1.6 Estadísticas básicas	3
1.7 Archivos CSV	4
 2. Decisiones técnicas y arquitectura	 4
2.1 Estructura de módulos	4
2.2 Diagrama de flujo	5
2.3 Capturas de ejecución	6
 3. Dificultades y conclusiones	 7
4. Bibliografía y webgrafía	8
5. Enlaces del proyecto	8

1. Marco teórico

A continuación se describen los conceptos de programación aplicados en el desarrollo del sistema, junto con la forma concreta en que cada uno se utilizó en el proyecto.

1.1 Listas

Una lista es una estructura de datos que permite almacenar una colección ordenada y modificable de elementos. En este proyecto, cada llamada a la base de datos devuelve una lista de países, donde cada país está representado internamente como un diccionario. Sobre esa lista se aplican operaciones de recorrido, filtrado (mediante comprensión de listas) y ordenamiento.

1.2 Diccionarios

Un diccionario almacena pares clave-valor, lo que resulta ideal para representar una entidad con atributos con nombre. Cada país se modela como un diccionario con las claves 'nombre', 'poblacion', 'superficie' y 'continente'. Esta elección permite acceder a cada atributo de forma directa y legible (por ejemplo, pais['poblacion']), en lugar de depender de índices numéricos como ocurriría con una tupla o lista posicional.

1.3 Funciones

Las funciones permiten dividir el programa en unidades reutilizables, cada una responsable de una única tarea. El proyecto aplica este principio separando el código en cuatro módulos: database.py concentra la lectura y escritura del archivo CSV; manager.py contiene la lógica de negocio (filtros, ordenamientos, estadísticas); ui.py se encarga de la interacción con el usuario por consola; y main.py orquesta el menú principal. Esta modularización facilita las pruebas, la lectura del código y el mantenimiento.

1.4 Condicionales

Las estructuras condicionales (if/elif/else) permiten que el programa tome decisiones según el estado de los datos. Se utilizan, por ejemplo, para validar que los campos ingresados no estén vacíos, para verificar que los valores numéricos sean válidos antes de convertirlos, para comprobar que el mínimo de un rango no sea mayor que el máximo, y para dirigir la ejecución hacia la opción de menú seleccionada por el usuario.

1.5 Ordenamientos

Ordenar una colección consiste en reorganizar sus elementos según un criterio determinado. El proyecto utiliza la función sorted() de Python, indicando mediante el parámetro key qué atributo del país se debe usar como criterio de comparación (nombre, población o superficie) y el parámetro reverse para alternar entre orden ascendente y descendente. Python implementa internamente un algoritmo de ordenamiento estable (Timsort), por lo que no fue necesario programar un algoritmo de ordenamiento manual.

1.6 Estadísticas básicas

Sobre el conjunto de países cargados se calculan indicadores simples: el país con mayor y menor población (mediante las funciones max() y min() con una función clave), el promedio de población y de superficie (sumando los valores con sum() y dividiendo por la cantidad de elementos) y la cantidad de países por continente (acumulando un conteo en un diccionario auxiliar).

1.7 Archivos CSV

CSV (Comma-Separated Values) es un formato de texto plano para representar datos tabulares, donde cada línea es un registro y sus campos están separados por comas. El proyecto utiliza el módulo estándar csv de Python, en particular las clases DictReader y DictWriter, que permiten leer y escribir cada fila como un diccionario asociando directamente cada valor con el nombre de su columna, evitando así el manejo manual de índices de columna.

2. Decisiones técnicas y arquitectura

El sistema se organizó siguiendo una arquitectura en capas, separando responsabilidades en cuatro módulos independientes. Esta decisión responde al principio de que cada función (y cada archivo) debe tener una única responsabilidad, lo que facilita la localización de errores y la incorporación de nuevas funcionalidades sin afectar al resto del sistema.

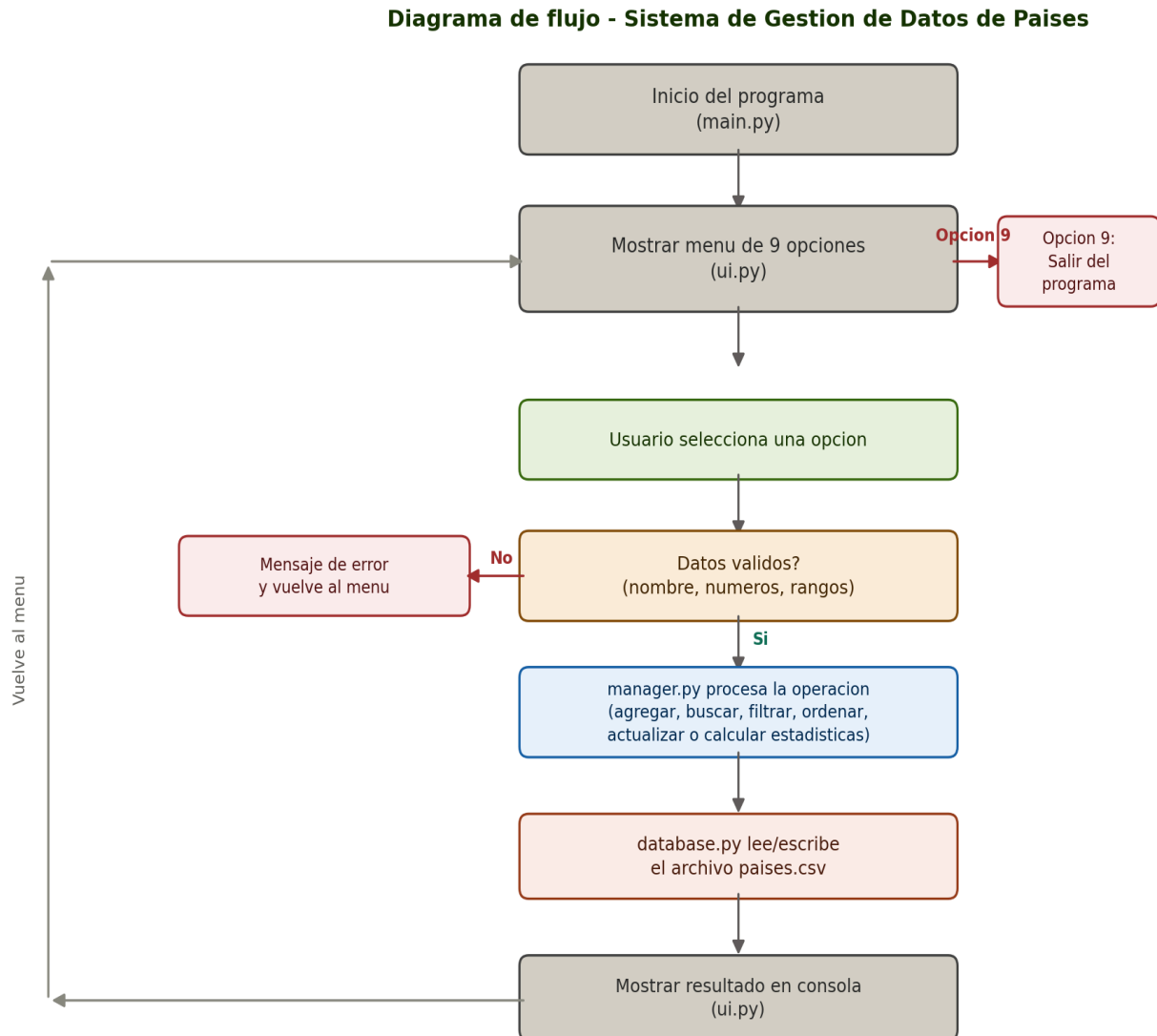
2.1 Estructura de módulos

Módulo	Responsabilidad
main.py	Punto de entrada del programa; muestra el menú y despacha la opción elegida.
ui.py	Lee datos por consola, valida entradas simples y presenta los resultados al usuario.
manager.py	Lógica de negocio: inserción, actualización, búsqueda, filtros, ordenamientos y estadísticas.
database.py	Persistencia: lectura y escritura del archivo paises.csv.

Esta separación implica que, por ejemplo, cambiar el formato de almacenamiento (de CSV a otra fuente de datos) solo requeriría modificar database.py, sin afectar la lógica de negocio ni la interfaz de usuario.

2.2 Diagrama de flujo

El siguiente diagrama resume el flujo principal de operaciones del sistema, desde el inicio del programa hasta la persistencia de los datos en el archivo CSV:



2.3 Capturas de ejecución

A continuación se muestran ejemplos reales de ejecución del programa en consola.

Menú principal y cálculo de estadísticas:

```
python3 main.py

--- GESTIÓN DE PAÍSES ---
1. Agregar País
2. Actualizar País
3. Buscar País por nombre
4. Filtrar por Continente
5. Filtrar por rango de Población
6. Filtrar por rango de Superficie
7. Ordenar países
8. Mostrar Estadísticas
9. Salir
Seleccione una opción: 8

--- Estadísticas ---
Mayor población: Brasil (213993437)
Menor población: Argentina (45376763)
Promedio de población: 117079875.00
Promedio de superficie: 3007791.00 km2
Cantidad por continente:
- America: 2
- Asia: 1
- Europa: 1
```

Filtrado por continente:

```
python3 main.py

Seleccione una opción: 4
Ingrese el continente a filtrar: America
- Argentina: Población 45376763, Superficie 2780400 km2, America
- Brasil: Población 213993437, Superficie 8515767 km2, America
```

Ordenamiento por población descendente:

```
python3 main.py

Seleccione una opción: 7
Ordenar por: 1. Nombre 2. Población 3. Superficie
Seleccione una opción: 2
¿Ascendente o descendente? (A/D): D
- Brasil: Población 213993437, Superficie 8515767 km2, America
- Japon: Población 125800000, Superficie 377975 km2, Asia
- Alemania: Población 83149300, Superficie 357022 km2, Europa
- Argentina: Población 45376763, Superficie 2780400 km2, America
```

3. Dificultades y conclusiones

La parte que más trabajo me llevó fue el diseño de las funciones de filtrado y ordenamiento en `manager.py`, especialmente decidir cómo estructurar los criterios de ordenamiento (por nombre, población o superficie, ascendente o descendente) sin duplicar código. Lo resolví usando funciones lambda como clave de ordenamiento, lo que permitió reutilizar una sola función genérica para los tres campos.

Otro punto que me demandó atención fue el manejo del archivo CSV: evitar que el programa se rompiera si el archivo no existía o si alguna fila tenía datos corruptos. Lo solucioné validando la existencia del archivo antes de leerlo y descartando silenciosamente las filas que no podían convertirse a los tipos esperados.

Durante las pruebas encontré errores relacionados principalmente con la conversión de datos ingresados por el usuario (por ejemplo, ingresar texto donde se esperaba un número), que corregí agregando validaciones con `try/except` en las funciones de la interfaz de usuario. También detecté que era necesario separar claramente la lectura/escritura de datos (`database.py`) de la lógica de negocio (`manager.py`) para que el código fuera más fácil de depurar.

Los conceptos que más reforcé fueron el uso de listas de diccionarios como estructura de datos, la modularización de funciones (una función, una responsabilidad) y el manejo de archivos con la librería `CSV`.

Si tuviera que empezar de nuevo, probablemente cargaría los datos una sola vez en memoria al iniciar el programa y trabajaría sobre esa copia, guardando los cambios sólo cuando fuera necesario, en lugar de leer el CSV completo en cada operación.

Al haber trabajado de forma individual, la organización consistió en dividir el desarrollo por capas: primero la persistencia de datos, luego la lógica de negocio y por último la interfaz de consola, probando cada módulo antes de avanzar con el siguiente.

4. Bibliografía y webgrafía

- [Python Software Foundation — Documentación oficial de Python 3 \(estructuras de datos\)](#)
- [Python Software Foundation — Módulo csv \(lectura y escritura de archivos CSV\)](#)
- [Python Software Foundation — Funciones integradas sorted\(\), max\(\), min\(\), sum\(\)](#)
- [Python Software Foundation — Definición de funciones \(def\)](#)

5. Enlaces del proyecto

Repositorio en GitHub: [AgusStein99/TPI_PROG_1: TPI Para Examen Final](#)